

CI/CD Pipeline - Cheat Sheet

CI/CD pipelines automate the path from code commit to production. Push a branch, watch a sequence of jobs build, test, scan, and deploy your changes. The pipeline is the engineering team's shared safety net and force multiplier.

Done well, you ship multiple times a day with confidence. Done poorly, the pipeline is a slow, flaky bottleneck that the team works around.

CI vs CD vs CD

Three related concepts, often conflated.

Term	What it means
Continuous Integration (CI)	Every commit gets built and tested automatically. Catches integration issues early.
Continuous Delivery (CD)	Every successful build is ready to deploy. A human triggers the release.
Continuous Deployment (CD)	Every successful build deploys to production automatically. No human gate.

CI is the foundation. Delivery and deployment differ only on whether humans approve releases. Most teams sit somewhere in between: CI for everything, automatic deploys to dev and staging, manual approval for prod.

Pipeline Stages

A typical pipeline runs these stages in order. Failure at any stage stops the rest.

```
commit -> build -> unit tests -> integration tests -> static analysis
      -> security scan -> package -> deploy to staging -> e2e tests
      -> manual approval -> deploy to prod -> smoke tests
```

Most pipelines tune this for their codebase, but the shape stays the same: fast feedback first, expensive checks later, deployment last.

Common Tools

Tool	Hosting	Pipeline definition
GitHub Actions	GitHub-hosted or self-hosted	YAML in <code>.github/workflows/</code>
GitLab CI/CD	GitLab-hosted or self-hosted	YAML in <code>.gitlab-ci.yml</code>
Jenkins	Self-hosted (mostly)	Jenkinsfile (Groovy DSL)
CircleCI	SaaS	YAML in <code>.circleci/config.yml</code>
Bitbucket Pipelines	SaaS	YAML in <code>bitbucket-pipelines.yml</code>
Azure DevOps	SaaS or self-hosted	YAML or classic UI
Google Cloud Build	SaaS	YAML in <code>cloudbuild.yaml</code>
Tekton / Argo Workflows	Kubernetes-native	Kubernetes CRDs
Drone CI	Self-hosted	YAML

GitHub Actions has the largest mindshare today, thanks to GitHub's reach. Jenkins is still common in enterprises with a lot of legacy infrastructure.

Triggers

A pipeline runs when something happens. Common triggers:

- **Push to a branch:** every commit to `main` or any branch.
- **Pull request:** open, reopen, or update.
- **Tag:** typically `v1.2.3` for releases.
- **Manual:** a button in the UI or a `workflow_dispatch` call.
- **Schedule:** cron-style timer (nightly builds, weekly security scans).
- **Webhook from another system:** another pipeline finishes, an external event fires.

GitHub Actions example:

```
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
  workflow_dispatch:    # manual trigger
  schedule:
    - cron: '0 2 * * *' # nightly at 02:00 UTC
```

Pipeline as Code

The pipeline definition lives in the repo, versioned with the code it ships. Changes go through code review. Rollbacks are git reverts.

GitHub Actions example

```
# .github/workflows/ci.yml
name: CI

on:
  push:
    branches: [main]
  pull_request:

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-java@v4
        with:
          java-version: '21'
          distribution: temurin
          cache: maven

      - name: Build and test
        run: ./mvnw verify

      - name: Upload coverage
        uses: codecov/codecov-action@v4

  docker:
    needs: build
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main'
    steps:
      - uses: actions/checkout@v4

      - name: Build image
        run: docker build -t myapp:${{ github.sha }} .

      - name: Push to registry
        run: |
          echo "${{ secrets.REGISTRY_PASS }}" \
            | docker login -u "${{ secrets.REGISTRY_USER }}" \
              --password-stdin registry.example.com
          docker push registry.example.com/myapp:${{ github.sha }}
```

GitLab CI example

```
# .gitlab-ci.yml
stages:
  - build
  - test
  - deploy

build:
  stage: build
  image: maven:3.9-eclipse-temurin-21
  script:
    - mvn package -DskipTests
  artifacts:
    paths:
      - target/*.jar

test:
  stage: test
  image: maven:3.9-eclipse-temurin-21
  script:
    - mvn verify

deploy-staging:
  stage: deploy
  script:
    - kubectl apply -f k8s/staging/
  only:
    - main
```

Build Stage

Compile the source. Generate any code (protobuf, OpenAPI clients). Resolve dependencies.

Best practices:

- **Cache dependencies.** A Maven or npm install can be the slowest step. Cache the local repository or `node_modules`.
 - **Pin tool versions.** A Maven 3.8 build that suddenly runs on Maven 3.9 can break in surprising ways.
 - **Fail fast.** Stop the pipeline at the first failure. Don't burn 20 minutes of e2e tests when the unit tests already failed.
 - **Reproducible builds.** Same input, same output. Helps with debugging and audit trails.
-

Test Stages

Layered, fastest first.

Stage	Purpose	Typical time
Unit tests	One class or function in isolation	Seconds
Integration tests	Two or more components together (DB, HTTP)	Minutes
Contract tests	API consumer-provider compatibility	Minutes
End-to-end (e2e) tests	Full system through real APIs and UIs	Minutes to tens of minutes
Performance tests	Load, stress, soak	Minutes to hours
Security scans (SAST, DAST)	Find vulnerabilities	Minutes

Pipelines run unit tests on every PR. Integration and e2e tests may run on every PR or only on merges to `main`. Performance tests usually run nightly or pre-release.

Code Quality Gates

Block bad code from merging. Common gates:

Gate	What it checks
Linters	Style violations, common bugs (Checkstyle, ESLint, golangci-lint)
Formatter	Code formatting (Prettier, gofmt, Spotless)
Static analysis	Deeper issues (SpotBugs, ErrorProne, SonarQube)
Coverage threshold	New code has tests (JaCoCo, Codecov)
Security scan	Known CVEs in dependencies (Snyk, Dependabot, Trivy)
License check	No GPL where you can't use it (FOSSA, license-checker)
Secret scan	No API keys committed (gitLeaks, truffleHog)

A gate fails the build if its check fails. Block the merge button until the build passes.

Artifact Management

A pipeline produces artifacts: JARs, container images, npm packages, Helm charts. These need a registry.

Artifact	Registry
Java JARs / WARs	Maven Central, Nexus, Artifactory, GitHub Packages
Docker images	Docker Hub, ECR, GCR, GHCR, Harbor
npm packages	npm registry, GitHub Packages, Nexus
Helm charts	OCI registries (ECR, Artifactory), Chart Museum

Tag artifacts with the commit SHA or a semantic version. `latest` is fine in dev. Avoid `latest` in production.

Promote the same artifact through environments. Rebuilding per environment introduces drift between what was tested and what's running.

Deployment Strategies

How you replace the running version with a new one.

Rolling update

Replace pods or instances gradually. New ones come up, old ones drain. Kubernetes Deployments do this by default.

```
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxSurge: 1
    maxUnavailable: 0
```

Pros: zero downtime, simple.

Cons: both versions run side by side briefly. Backward compatibility matters.

Blue-green

Two full environments. Blue is live. Deploy the new version to green. Once green is healthy, switch the router from blue to green.

Pros: instant rollback by flipping back to blue, full pre-prod testing of the new version.

Cons: needs double the infrastructure during the switch.

Canary

Send a small percentage of traffic (5%, 25%) to the new version. Watch metrics. If they hold, increase the percentage. Otherwise, roll back.

Pros: catches problems before they hit everyone.

Cons: needs traffic-splitting infrastructure (service mesh, smart load balancer, Argo Rollouts, Flagger).

Recreate

Stop the old version, start the new one. Causes downtime.

Useful only when old and new can't run together (incompatible schema changes, single-instance apps).

Environments

Most teams have at least three.

Environment	Purpose
Dev / Development	Continuous deploys from <code>main</code> . Tolerate breakage.
Staging / Pre-prod	Production-like data and config. Final gate before prod.
Production	The real one. Promote from staging.

Some teams add `qa`, `uat`, `canary`, or `sandbox`. The exact list depends on the team.

Promotion flow:

```
commit -> CI -> build artifact -> deploy to dev
      -> integration tests pass -> deploy to staging
      -> manual approval -> deploy to prod
```

Each environment should be as close to production as possible. Different versions of DB, OS, or runtime cause “works in staging, breaks in prod” surprises.

Secrets Management

Never commit secrets to the repo. Pipelines need access to API keys, registry credentials, deployment tokens, signing keys.

Where they live:

- **CI platform secrets:** GitHub Actions secrets, GitLab CI/CD variables, Jenkins credentials.
- **External secret stores:** Vault, AWS Secrets Manager, GCP Secret Manager, 1Password.
- **Cloud workload identity:** the pipeline assumes a role (GitHub OIDC + AWS, GCP Workload Identity Federation). No long-lived keys.

Best practices:

- Treat the pipeline runner as a security boundary. A compromised pipeline can leak any secret it touches.
 - Scope secrets per environment. The dev deploy key shouldn’t work in prod.
 - Rotate regularly. Short-lived credentials via OIDC beat long-lived API keys.
 - Don’t echo secrets in logs. CI platforms try to mask them, but the masking can break if a secret appears mid-line.
-

Best Practices

- **Pipeline as code.** Definitions live in the repo and go through code review.
 - **Fast feedback first.** Unit tests at the front, slow e2e at the back.
 - **Fail fast.** A flaky 20-minute pipeline kills team velocity.
 - **Cache aggressively.** Dependencies, build outputs, Docker layers.
 - **One artifact, multiple environments.** Build once, promote everywhere.
 - **Version everything.** Code, infrastructure, pipeline config, artifacts.
 - **Treat deployments as boring.** Frequent small deploys beat rare big ones every time.
 - **Add manual approval for prod.** Or automate fully with good monitoring and rollback. Half-measures combine the worst of both.
 - **Monitor deploys.** Watch error rate, latency, and key business metrics. Auto-rollback if they go sideways.
 - **Make rollbacks one click.** `kubectl rollout undo`, redeploy the previous tag, flip the blue-green switch. Practice it before you need it.
-

Common Gotchas

- **Flaky tests in the pipeline.** A 95% reliable test that runs on every PR fails 1 in 20 builds. People learn to retry blindly. Fix the test or remove it.
 - **No artifact caching.** Pulling dependencies on every build wastes time and bandwidth.
 - **Building the same artifact multiple times.** If you build separately for each environment, dev and prod artifacts differ even when the source is identical.
 - **Secrets in logs.** A `set -x` or verbose error message can leak a token. Audit log output.
 - **Untracked pipeline changes.** A teammate edits the Jenkins job in the UI. Now nobody knows what the pipeline does. Use pipeline-as-code.
 - **Long-lived CI runners with broad access.** A compromised runner is a compromised cloud account. Use ephemeral runners and least-privilege IAM.
 - **Slow pipelines starve attention.** When CI takes 45 minutes, people stop watching it. Keep the critical path under 15 minutes.
 - **No staging that looks like prod.** A staging DB with 100 rows can't catch the query that times out on prod's 100 million rows.
 - **Manual deploys "just this once".** That's how prod and the pipeline drift apart. Always deploy through the pipeline.
 - **Coupled deploys.** A pipeline that ships 5 services at once means one failure rolls them all back. Decouple where you can.
 - **Forgetting database migrations.** Migrations need a separate strategy (backward-compatible changes, expand/contract pattern). Code rollback doesn't undo a schema change.
-

Quick Reference

Concept	Key fact
CI	Build and test every commit
Continuous Delivery	Every build is releasable, human approval
Continuous Deployment	Every build auto-deploys to prod
Pipeline as code	Definition in the repo, reviewed like code
Trigger	Push, PR, tag, schedule, manual, webhook
Build artifact	Tag with commit SHA, promote through environments
Test layers	Unit → integration → e2e, fast first
Quality gates	Lint, static analysis, coverage, security scan
Rolling update	Replace pods gradually (default in Kubernetes)
Blue-green	Two full environments, switch the router
Canary	Small traffic share to the new version, watch metrics
Secrets	Use the CI platform's store or an external manager. Never commit.
Rollback	One click. Practice before you need it.